SOFTWARE TESTING NOTES

FROM BASICS TO EXPERT

PLANNING → DESIGN → EXECUTION → DEFECTS → REPORTING → QUALITY

- Learn.
- Understand.
- Practice.
- Analyze.
- Excel.
- Succeed.

TOP 100

★ SOFTWARE TESTING INTERVIEW QUESTIONS ★

+

QUICK REVISION CHEAT SHEET

20 TOPICS COVERED

1. Software Testing Fundamentals
2. SDLC + STLC
3. Testing Principles + Levels
4. Test Types
5. Manual Testing
6. Test Case Design Techniques
7. Test Scenarios + Test Cases
8. Black Box Testing
9. White Box Testing
10. Regression + Retesting
11. Integration + System Testing
12. Functional + Non-Functional Testing
13. Defect/Bug Life Cycle
14. Severity + Priority
15. Test Automation Fundamentals
16. Selenium + Automation Concepts
17. API + Performance Testing
18. Agile Testing + CI/CD
19. Real-World Testing Problems
20. Top 100 Interview Qs + Cheat Sheet

Everything you need in one place!

> "Quality is not an act, it is a habit."
>
> Test. Learn. Improve. Deliver Quality.

★

WHAT YOU'LL GAIN

- ☑ Strong Conceptual Clarity
- ☑ Better Problem Solving
- ☑ Real Interview Preparation
- ☑ Quick Revision Anytime
- ☑ Practical Knowledge
- ☑ Confident & Job Ready
- ☑ Higher Accuracy in Coding & CS

Study Smart | Test Smart | Be Confident | Get Hired

1. Software Testing Fundamentals

What is Software Testing?

Software Testing is the process of evaluating and verifying that a software product or application does what it is supposed to do and does it correctly.

> **Note :**
> Testing shows the presence of defects, not the absence of defects.

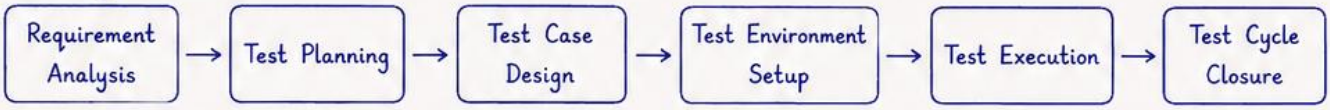
Objectives of Testing

- To find defects / bugs in the software.
- To ensure the software meets the requirements.
- To build confidence in the quality of the software.
- To provide information for decision making.
- To reduce the risk of software failure.

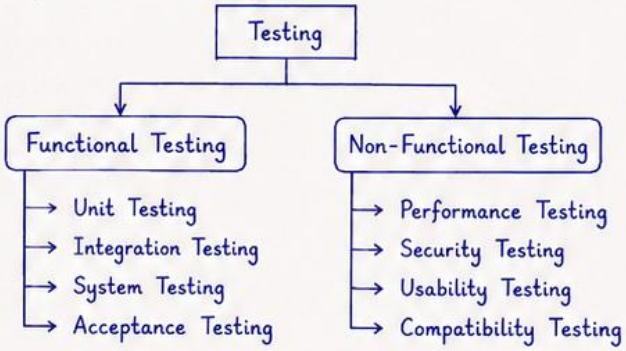
Testing Principles (ISTQB)

No.	Principle	No.	Principle
1	Testing shows the presence of defects.	6	Testing is context dependent.
2	Exhaustive testing is impossible.	7	Absence-of-errors fallacy.
3	Early testing saves time and money.	8	Testing is 100% measurement.
4	Defects cluster together.	9	The bug is never absent.
5	Pesticide paradox.	10	Good testing is risk based.

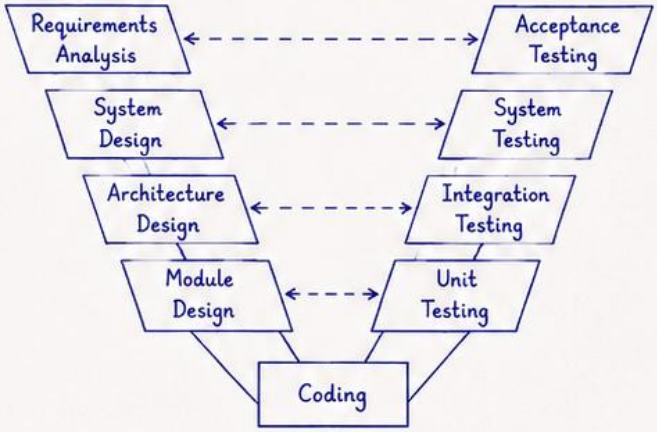
Testing Process



Types of Testing



V-Model (SDLC + Testing)



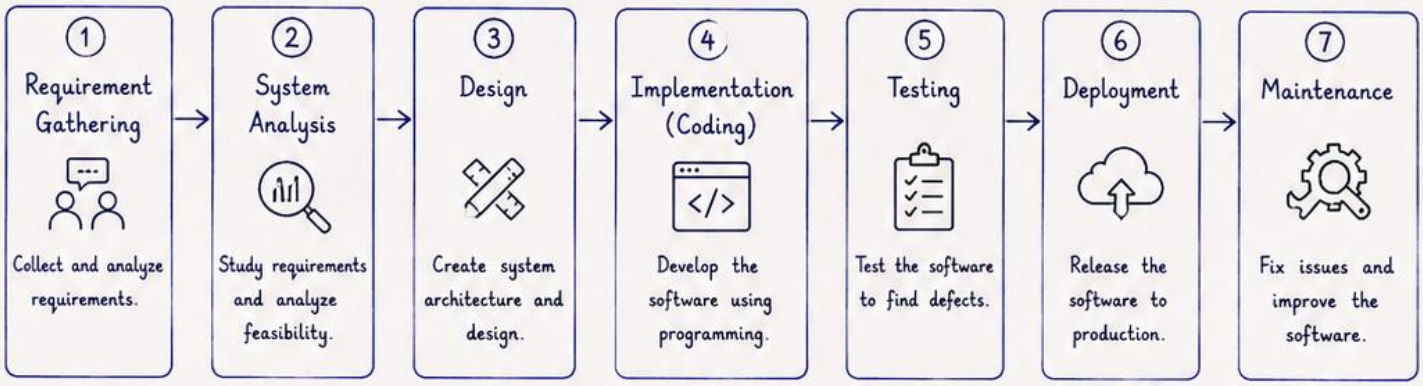
> **Key Takeaway ☆**
> Good testing leads to high quality software, happy users, and successful delivery. ☺

2. SDLC + STLC

@codedsoul_05

• SDLC (Software Development Life Cycle)

SDLC is a process followed by development teams to design, develop and deliver high quality software.



Key Points

→ SDLC focuses on building the right product.

→ Testing is one of the phases in SDLC.

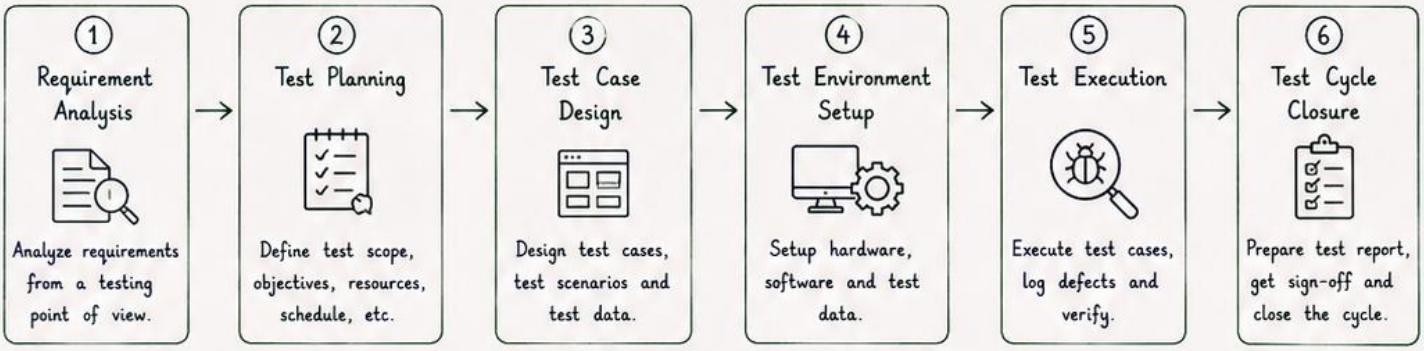
Note :

SDLC is development centric.

STLC is testing centric. ☺

• STLC (Software Testing Life Cycle)

STLC is a process followed by testers to ensure the quality of the software.



Key Points

→ STLC focuses on building the product right.

→ STLC starts when development is complete.

→ It ensures the quality of software before release.

SDLC vs STLC

	SDLC	STLC
Focus	Build the Right Product	Build the Product Right
Owner	Development Team	Testing Team
Starts	With Requirement Gathering	After/Parallel to SDLC
Goal	Create the Software	Ensure Quality
Activities	Development Activities	Testing Activities

Importance

- Helps in early defect detection.
- Ensures high quality software.
- Reduces cost and time.
- Improves customer satisfaction.
- Ensures timely delivery.

3. Testing Principles + Levels

@codedsoul_05

• Testing Principles

These principles guide every testing activity.

1. Testing shows the presence of defects, not the absence. Exhaustive testing is impossible.
2. Early testing saves time and money.
3. Defects cluster together.
4. Pesticide paradox – repeating the same tests will eventually not find new defects.
5. Testing is context dependent. (No 2 projects are same)
6. Absence-of-errors fallacy – a bug free system may still fail to meet the requirements.
7. Good testing is risk based.

Note :
Following these principles helps in building a high quality product and improves confidence in the software.

Key Takeaway

→ Testing helps to reduce risk.
→ Start testing early.
→ Focus on important and risky areas.
→ Quality is everyone's responsibility.

• Testing Levels

Testing is performed at different levels in a software development process.

1. Unit Testing
Testing individual units / components of the code.
Done by Developers.

2. Integration Testing
Testing the interaction between integrated units.
Done by Testers.

3. System Testing
Testing the complete system as a whole.
Done by Testers.

4. Acceptance Testing
Validating the system with business requirements.
Done by End Users / Clients.

5. Maintenance Testing
Testing after the system is released to ensure changes did not break existing features.
Done by Testers.

Levels Comparison

Level	Performed By	Objective	Focus Area	When Performed
Unit Testing	Developers	Verify individual unit works as intended.	Functions, Methods, Classes, Modules	During Development
Integration Testing	Testers	Verify interaction between units.	Interfaces, Data flow, APIs	After Unit Testing
System Testing	Testers	Verify the entire system meets the requirements.	Functional & Non-functional Requirements	After Integration Testing
Acceptance Testing	End Users / Clients	Validate if the system is ready for use.	Business Scenarios, User Workflows	After System Testing
Maintenance Testing	Testers	Ensure existing features work after changes.	Regression, Bug Fix Verification	After Deployment (Whenever Changes Are Made)

Quick View – Testing Levels in SDLC

Requirements → Development → Unit Testing → Integration Testing → System Testing → Acceptance Testing → Maintenance Testing

4. Test Types

@codedsoul_05

What are Test Types?

Test types are different ways of testing a software application to check its functionality, performance, security, usability, etc. They help ensure the software works as expected under different conditions.

Note :
Different test types focus on different aspects of the application. Using the right combination leads to better quality software. ☺

Functional Testing (Validates "WHAT" the system does)

Test Type	Purpose
Unit Testing	Tests individual units / components of the code.
Integration Testing	Tests the interaction between integrated units.
System Testing	Tests the complete system as a whole.
User Acceptance Testing (UAT)	Validates the system with business requirements. Done by end users.
Smoke Testing	Verifies the basic functionality of the build.
Sanity Testing	Tests specific functionality after minor changes.

Non-Functional Testing (Validates "HOW WELL" the system works)

Test Type	Purpose
Performance Testing	Checks speed, response time, stability under load.
Load Testing	Ensures the system performs well under expected load.
Stress Testing	Tests system behavior beyond normal load until failure.
Security Testing	Finds vulnerabilities and ensures data and system protection.
Usability Testing	Evaluates ease of use and user experience.
Compatibility Testing	Checks the application on different browsers, OS, devices, etc.

Structural Testing (Validates "HOW" the code works)

Test Type	Purpose
White Box Testing	Tests internal logic, code structure, paths, conditions.
Black Box Testing	Tests functionality without knowing internal code.
Gray Box Testing	Combination of White Box and Black Box testing.

Change Related Testing (Validates after changes)

Test Type	Purpose
Regression Testing	Ensures existing functionality works after changes.
Re-Testing	Verifies that reported defects are fixed.
Confirmation Testing	Checks if changes / fixes work in the live environment.

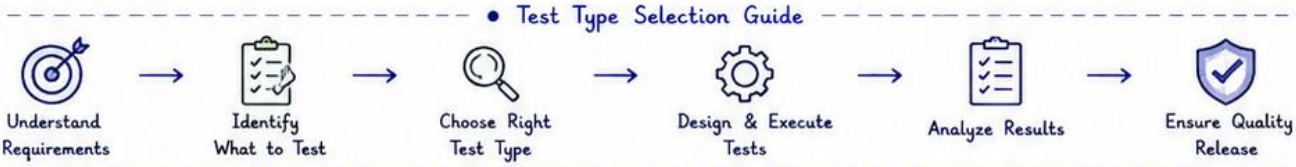
Other Important Test Types

- API Testing – Tests APIs for functionality, reliability.
- Database Testing – Validates data integrity and operations.
- Installation Testing – Checks installation / uninstallation.
- Recovery Testing – Ensures system recovers from crashes.
- Localization Testing – Validates application for different languages and regions.
- Accessibility Testing – Ensures the application is accessible to all users.

Functional vs Non-Functional Testing

Aspect	Functional Testing	Non-Functional Testing
Focus	What the system does	How well the system works
Examples	Unit, Integration, System, UAT	Performance, Security, Usability, Compatibility
Done By	Developers, Testers	Testers, Performance Engg. Security Experts
When	During SDLC	During & After Development
Goal	Verify functionality	Verify quality attributes

Test Type Selection Guide



5. Manual Testing

@codedsoul_05

What is Manual Testing?

Manual Testing is the process of testing a software application manually without the use of automation tools. Testers execute test cases step-by-step and compare the actual results with the expected results.

Note :
Manual testing is essential to find usability issues, visual issues and to validate the application from end user perspective. ☺

Key Characteristics

Requires human intelligence and observation.	Involves exploratory testing and ad-hoc testing.	Test cases are executed manually step-by-step.	Effective for usability, UI, and real-world scenarios.	Helps in finding issues that automation may miss.

Advantages

- ✓ No coding knowledge required.
- ✓ Cost effective for small projects.
- ✓ Easy to learn and implement.
- ✓ Best for UI, usability and ad-hoc testing.
- ✓ Can handle unpredictable test scenarios.
- ✓ Helps in exploratory testing.

Disadvantages

- ✗ Time consuming for repetitive tasks.
- ✗ Prone to human error.
- ✗ Difficult to re-execute large test suites.
- ✗ Not suitable for performance and load testing.
- ✗ Hard to generate test reports without tools.

When to Use Manual Testing?

- When the application is new.
- When requirements are unstable.
- For short-term or small projects.
- When UI/UX testing is important.
- When exploratory or ad-hoc testing is needed.
- When automation is not feasible or cost-effective.

Manual Testing Activities

1. **Requirement Analysis** — Understand the requirements and expected behavior.
2. **Test Planning** — Plan the testing scope, resources, schedule and approach.
3. **Test Case Design** — Create test cases, test scenarios and test data.
4. **Test Case Execution** — Execute test cases manually and record results.
5. **Defect Reporting** — Log defects with proper steps, severity and priority.
6. **Retesting & Regression** — Verify the defects are fixed and check for side effects.
7. **Test Closure** — Prepare test summary report and close the testing.

Types of Manual Testing (Common)

Functional Testing	Validates the functionality of the application as per requirements.
Non-Functional Testing	Validates non-functional aspects like performance, security, usability, etc.
UI Testing	Checks the look and feel, layout, fonts, colors, alignment, etc.
Usability Testing	Evaluates the ease of use and user experience.
Exploratory Testing	Simultaneous learning, test design and test execution.

Good Manual Tester Should...

- ✓ Have good analytical skills
- ✓ Be detail oriented
- ✓ Have good communication skills
- ✓ Be patient and curious
- ✓ Understand the domain well
- Have good documentation skills
- Be able to think like an end user
- Have basic technical knowledge

Manual Testing vs Automation Testing

Aspect	Manual Testing	Automation Testing
Execution	Manual by Tester	Using Automation Tools
Tool	Not Required	Required
Cost	Low Initial Cost	High Initial Cost
Best For	Small, UI, Ad-hoc, Exploratory	Large, Repetitive, Regression
Time	More Time	Less Time (after setup)
Human Involvement	High	Low
Flexibility	High	Low

Key Takeaway
Manual testing ensures the application works correctly from the end user's perspective and helps deliver a high quality product. ☺

6. Test Case Design Techniques

@codedsoul_05

What is Test Case Design Technique?

Test case design techniques are systematic methods used to identify test cases for verifying the functionality and reliability of the software. They help in designing effective and efficient test cases with good coverage.

Note :
Good test case design helps in finding defects early, reduces effort and cost, and improves test coverage.

Objectives

- [x] To identify defects.
- [x] To maximize test coverage.
- [x] To minimize the number of test cases.
- [x] To save time and effort.
- [x] To ensure quality of the application.

Classification of Techniques

1. Black Box Techniques
Based on requirements and functional specifications.
Tester has no knowledge of internal code.

2. White Box Techniques
Based on internal structure and code.
Tester has knowledge of internal code.

Common Test Case Design Techniques

Technique	Type	Description	Example	When to Use
1. Equivalence Partitioning	Black Box	Divides input data into valid and invalid partitions. Test one value from each partition.	Age: 1–18 (Invalid), 19–60 (Valid), 61–100 (Invalid)	When input domains are large.
2. Boundary Value Analysis	Black Box	Focuses on the boundary values of input domains.	Range: 1–100 Test: 0, 1, 2, 99, 100, 101	When boundary errors are likely.
3. Decision Table Testing	Black Box	Used when business logic has complex combinations of conditions and actions.	Multiple conditions (C1, C2) and actions (A1, A2)	When there are multiple business rules.
4. State Transition Testing	Black Box	Tests the behavior of the system based on state changes.	Login -> Logged In -> Logout -> Logged Out	When the system changes states.
5. Use Case Testing	Black Box	Based on user scenarios (use cases).	Use case: User Login -> Steps -> Expected result	When requirements are defined in use cases.
6. Error Guessing	Black Box	Based on experience to guess where errors may occur.	Login with empty fields, special characters, etc.	When there is time constraint.
7. Statement Coverage	White Box	Ensures every statement in the code is executed at least once.	Cover all lines of code in the program.	To check overall code execution.
8. Branch (Decision) Coverage	White Box	Ensures every branch (True/False) is executed at least once.	If condition -> True path & False path	To test decision points in the code.
9. Path Coverage	White Box	Ensures all possible independent paths are executed.	Multiple paths through the code.	For complex logic and high-risk modules.
10. Loop Testing	White Box	Tests loops with zero, one, many and maximum iterations.	Loop runs 0, 1, n-1, n, n+1 times.	When loops are present in the code.

Black Box vs White Box Techniques

Aspect	Black Box	White Box
Knowledge	No knowledge of internal code	Knowledge of internal code
Focus	Input & Output	Code, Logic & Paths
Techniques	EP, BVA, DT, ST, Use Case, Error Guessing	Statement, Branch, Path, Loop Coverage
Tester	Functional Tester	Developer / Technical Tester
Purpose	Validate functionality	Validate code logic & structure

How to Choose a Technique?

- ✓ Understand requirements and functionality.
- ✓ Identify input conditions and business rules.
- ✓ Analyze risks and critical areas.
- ✓ Select appropriate technique (or combination).
- ✓ Design test cases and review for completeness.

Key Takeaway

Use the right technique at the right time for better test coverage, early defect detection, and efficient testing.

Understand Requirements → Choose Right Technique → Design Test Cases → Review & Execute → Find Defects & Ensure Quality

7. Test Scenarios + Test Cases

@codedsoul_05

What are Test Scenarios and Test Cases?

Test Scenario: High level description of a feature or functionality from the end user perspective.

Test Case: Detailed step-by-step description of how a particular scenario will be tested.

Note :
Scenarios describe WHAT to test and Test Cases describe HOW to test it. Good scenarios lead to good test cases and better test coverage. ☺

Test Scenario

- Written from end user perspective.
- High level.
- Describes what should be tested.
- Not very detailed.
- Helps in identifying test conditions.

Test Case

- Written from tester perspective.
- Detailed and step-by-step.
- Describes how to test.
- Has pre-conditions, test data, expected results.
- Used for test execution and defect reporting.

Example

Feature: User Login

Username
Password
Login

⇒

Test Scenarios (High Level)

1. Verify user can login with valid credentials.
2. Verify user cannot login with invalid credentials.
3. Verify error message is displayed for invalid login.
4. Verify "Remember Me" functionality.
5. Verify user is able to logout successfully.

⇒

Test Cases (Detailed)

- TC_01: Login with valid username & password
- TC_02: Login with invalid username
- TC_03: Login with invalid password
- TC_04: Login with empty username
- TC_05: Login with empty password
- TC_06: Login with both fields empty
- TC_07: Login with Remember Me checked
- TC_08: Logout from the application

How Test Scenarios are converted into Test Cases?

Understand Requirements → Identify Test Scenarios → Break Scenarios into Test Conditions → Design Test Cases for Each Condition → Review & Refine Test Cases

Test Case Template

Test Case ID	Test Case Name	Test Scenario ID	Priority	Author	Date
TC_01	Login with valid credentials	TS_01	High	Tester	20-May-2025
Module	Login Module				
Pre-Conditions	User is on the login page. User is registered.				
Test Data	Username: user1 Password: Pass@123				
Step No.	Test Steps	Test Data	Expected Result	Actual Result	Status
1	Enter valid username	user1	Username should be accepted		
2	Enter valid password	Pass@123	Password should be accepted		
3	Click on Login button	–	User should be logged in and redirected to Dashboard		
4	Verify user is on Dashboard	–	Dashboard is displayed with user name		

Status: Pass (✓) Fail (✗) Blocked (II) Not Executed (–)

Good Test Scenarios are...

- ✓ Clear and concise
- ✓ Cover positive and negative flows
- ✓ Based on user goals and business rules
- ✓ Independent and testable

Good Test Cases are...

- ✓ Clear, simple and easy to execute
- ✓ Have clear pre-conditions and test data
- ✓ Have expected results
- ✓ Reusable and maintainable

Key Takeaway

Scenarios help us think from a user perspective and cover all important conditions. Test cases help us verify each condition in a structured and repeatable way. ☺

From Scenario to Execution (Quick View)

Requirements → Test Scenarios → Test Conditions → Test Cases → Test Execution → Results & Defects → Closure

8. Black Box Testing

@ codedsoul_05

What is Black Box Testing?

Black Box Testing is a software testing technique where the internal structure or implementation of the application is not known to the tester.

Tester focuses on verifying the functionality based on requirements and specifications.

Note :
In Black Box Testing, we test the application from the outside without knowing how it works inside. ☺

Characteristics

- ✓ Focuses on input and output.
- ✓ No knowledge of internal code/structure required.
- ✓ Based on requirements and specifications.
- ✓ Can be performed by testers, developers or end users.
- ✓ Used to test functionality, usability and performance.
- ✓ Helps in finding missing or incorrect functionality.

Limitations

- ✗ Cannot test internal code, logic and paths.
- ✗ Low code coverage (compared to White Box Testing).
- ✗ Cannot identify unused code.
- ✗ Error localization is difficult.
- ✗ Requires well written requirements and test cases.

When to Use?

- When the internal structure of the system is not known.
- When we want to test the functionality as per requirements.
- When end user perspective testing is required.
- When acceptance and system testing is needed.
- Helps you want to find missing functionalities.

Types of Black Box Testing

Functional Testing	Non-Functional Testing	Regression Testing	Integration Testing	System Testing	Acceptance Testing
Validates functional requirements.	Validates performance, usability, security, etc.	Ensures existing functionality works after changes.	Validates interaction between modules.	Validates the complete system as a whole.	Validates system from end user perspective.

Common Black Box Test Design Techniques

Technique	Description	Example	When to Use
1. Equivalence Partitioning	Divides input data into valid and invalid partitions.	Age: 1–18 (Invalid), 19–60 (Valid), 61–100 (Invalid)	When input domains are large.
2. Boundary Value Analysis	Focuses on boundary values of input domains.	Range: 1–100 Test: 0, 1, 2, 99, 100, 101	When boundary errors are likely.
3. Decision Table Testing	Used for complex business rules with multiple conditions.	Conditions: C1, C2 Actions: A1, A2	When there are multiple business rules.
4. State Transition Testing	Tests system behavior due to state changes.	Login → Logged In → Logout → Logged Out	When the system has different states.
5. Use Case Testing	Tests based on user scenarios (use cases).	Use Case: User Registration → Steps → Expected Result	When requirements are defined in use cases.
6. Error Guessing	Based on experience to guess where errors may occur.	Empty fields, special characters, wrong formats, etc.	When time is limited and risk is high.
7. Checklist Based Testing	Uses a checklist of items to ensure nothing is missed.	UI Checklist, Functionality Checklist, Security Checklist	For sanity, smoke or quick testing.

Black Box vs White Box Testing

Aspect	Black Box Testing	White Box Testing
Knowledge	No knowledge of internal code	Full knowledge of internal code
Focus	Input & Output	Code, Logic & Paths
Performed By	Testers, End Users	Developers, Testers
Techniques	EP, BVA, DT, ST, Use Case, Error Guessing	Statement, Branch, Path, Loop Testing
Test Level	System, Acceptance, Integration	Unit, Integration
Purpose	Validate functionality	Validate code logic & structure

Key Takeaway

- ✓ Black Box Testing ensures the application works as per requirements from the user's perspective.
- ✓ It helps in finding functional issues, missing features and incorrect behavior.
- ✓ Good test design techniques help in creating effective and efficient test cases with high coverage. ☺

Black Box Testing Process (Quick View)

Understand Requirements → Identify Test Conditions → Design Test Cases → Execute Test Cases → Report Defects → Verify & Close Defects → Test Summary & Closure

9. White Box Testing

@codedsoul_05

What is White Box Testing?

White Box Testing is a testing technique in which the internal structure, code, logic and paths of the application are known to the tester.
Tester designs test cases based on the internal code structure to ensure all statements, branches, paths and conditions are executed at least once.

Note :
Also called Glass Box Testing or Structural Testing. It is usually performed by developers or testers with programming knowledge. ☺

Characteristics

- Focuses on internal code structure.
- Requires programming knowledge.
- Test cases are designed based on code logic and paths.
- High code coverage.
- Helps in finding logical errors, dead code and security issues.

Limitations

- Time consuming and costly.
- Not suitable for large legacy applications.
- Requires skilled testers with coding knowledge.
- Does not ensure functionality from user perspective.
- May miss issues in requirements or missing functionality.

When to Use?

- When the internal structure of the system is available.
- During unit testing and component testing.
- To find logical errors and security vulnerabilities.
- To achieve high code coverage.
- When performance and reliability are critical.

Types of White Box Testing

Unit Testing	Integration Testing	Component Testing	System Testing	Security Testing
Tests individual functions or methods.	Tests the interaction between modules.	Tests individual components or classes.	Tests the complete system structure.	Tests internal code for security vulnerabilities.

White Box Test Design Techniques (Based on Code Structure)

Technique	Description	Example / Focus	When to Use
1. Statement Coverage	Ensures every statement in the code is executed at least once.	Execute all lines of code. Focus: All statements	Basic level of coverage.
2. Branch Coverage	Ensures every branch (True/False) is executed at least once.	If condition → True path and False path. Focus: All branches	To test decision points.
3. Condition Coverage	Ensures every individual condition in a decision evaluates to True and False.	If (A && B) → A=True, A=False, B=True, B=False Focus: All conditions	When decisions have multiple conditions.
4. Condition/Decision Coverage	Ensures both the decision and each condition are covered.	Decision + All conditions Focus: Decision + Conditions	For complex logical expressions.
5. Path Coverage	Ensures all possible independent paths in the code are executed.	All paths from start to end. Focus: All paths	For critical and complex applications.
6. Loop Testing	Tests loops with zero, one, many and maximum iterations.	0 times, 1 time, n times, max times. Focus: Loop boundaries	When loops are present in the code.

White Box vs Black Box Testing

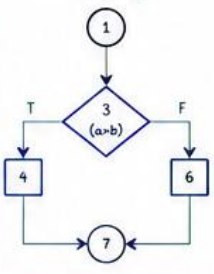
Aspect	White Box Testing	Black Box Testing
Knowledge	Full knowledge of internal code	No knowledge of internal code
Focus	Code, Logic, Paths, Structure	Input, Output, Functionality
Performed By	Developers, Testers	Testers, End Users
Techniques	Statement, Branch, Path, Loop, Condition	EP, BVA, DT, ST, Use Case, Error Guessing
Test Level	Unit, Integration, Component	System, Acceptance
Purpose	Validate code logic and structure	Validate functionality
Coverage	High (Code Coverage)	Functional Coverage

Example: Code & Path Coverage

Sample Code

```
1. int max(int a, int b)
2. {
3.     if (a > b)
4.         return a;
5.     else
6.         return b;
7. }
```

Flow Graph



Independent Paths

- Path 1: 1-3(T)-4-7
- Path 2: 1-3(F)-6-7

To achieve 100% path coverage, both paths must be tested. ☺

Key Takeaway

- White Box Testing ensures the internal code works as expected.
- It helps in finding logical errors, dead code and security issues.
- It complements Black Box Testing and is essential for high quality.

☺

White Box Testing Process (Quick View)

Understand Code → Identify Paths & Conditions → Design Test Cases → Execute Test Cases → Find Defects → Fix & Retest

10. Regression + Retesting

@ codedsoul_05

What are Regression Testing and Retesting?

Regression Testing: Re-running test cases to ensure that recent changes (bug fixes, enhancements, or code changes) have not introduced new defects in existing functionality.

Retesting: Re-running failed test cases after a defect has been fixed to verify that the fix works as expected.

> **Note :**
> Retesting checks the fix.
> Regression testing checks the side effects of the changes. ☺

Regression Testing



- Ensures existing functionality still works.
- Focuses on the impact of changes.
- Can be Partial or Complete.
- Helps in maintaining overall system stability.
- Prevents "bug fixing → bug breaking" cycle.

Retesting



- Ensures the defect that was fixed is resolved.
- Focuses only on failed test cases.
- Usually smaller in scope than regression testing.
- Confirms the fix without verifying other areas.
- First level of testing after a defect fix.

Key Differences

Aspect	Retesting	Regression Testing
Purpose	To verify that a specific defect is fixed.	To ensure recent changes have not affected existing functionality.
Scope	Limited to failed test cases.	Wider – includes related and unaffected areas.
When Performed	After a defect fix is deployed.	After any change in the application (fix, enhancement, config change).
Test Cases Involved	Only previously failed test cases.	Selected or all relevant test cases (based on impact).
Focus	Confirms the fix.	Confirms no side effects.
Execution Time	Less time consuming.	More time consuming.
Example	Verify login works after password bug is fixed.	Verify login, dashboard, reports, and other modules still work.

Types of Regression Testing

Complete Regression Testing	All test cases are re-executed. High time and resource consuming.
Partial Regression Testing	Only selected test cases that are impacted are executed.
Selective Regression Testing	Only test cases related to the changed modules are executed.
Prioritized Regression Testing	Test cases are prioritized based on risk and business impact.
Automated Regression Testing	Regression test cases are automated to save time and effort.

Types of Retesting

Immediate Retesting	Retesting done immediately after the fix is available.
Deferred Retesting	Retesting done after all the fixes are deployed together.
Confirmation Retesting	Developer's fix is verified by tester.
Selective Retesting	Only relevant failed test cases are re-tested.

Best Practices

- ✓ Analyze the impact of changes before selecting test cases.
- ✓ Maintain and update test cases regularly.
- ✓ Automate regression test cases for repeated areas.
- ✓ Prioritize test cases based on risk and business criticality.
- ✓ Retest immediately after fix and regression after confirmation. ☺

Regression & Retesting Flow (Quick View)



11. Integration + System Testing

@codedsoul_05

What are Integration and System Testing?

Integration Testing: Testing of combined units or modules to verify the interaction between them.

System Testing: Testing the complete and integrated application as a whole to verify it against the specified requirements.

> **Note :**
> Integration testing focuses on interfaces between modules. System testing focuses on the entire system from end-user perspective.

Integration Testing



- Performed after unit testing.
- Tests interfaces between modules.
- Identifies defects in data flow, control flow, and interface mismatches.
- Ensures modules work together as expected.
- Can be done in incremental builds.

System Testing



- Performed after integration testing.
- Tests the complete system as a whole.
- Validates the system against requirements.
- End-to-end testing from user perspective.
- Environment close to production.

Key Differences

Aspect	Integration Testing	System Testing
Objective	To test interaction between integrated modules.	To test the complete system against requirements.
Scope	Narrow – within combined modules.	Wide – entire system.
Test Level	Mid level.	High level.
Performed By	Testers / Developers	Independent Test Team
Test Basis	Design, Interface specs	SRS (Software Requirement Specification)
Focus	Interfaces, Data flow, Control flow	Functionality, Usability, Performance, Security, etc.
Test Environment	Partial / Simulated	Production-like environment
Test Data	Specific to module interactions	Realistic, end-to-end scenarios
When Performed	After unit testing and during build	After integration testing
Example	Testing API communication between Login and User Service.	Testing the entire e-commerce application from login to order placement.

Types of Integration Testing

Type	Description
Big Bang Integration	All modules are integrated at once and tested. Simple but hard to identify defects.
Top Down Integration	Integration starts from top (main) module and moves down to lower level modules.
Bottom Up Integration	Integration starts from lowest level modules and moves up to top module.
Sandwich / Hybrid Integration	Combination of Top Down and Bottom Up integration.

Types of System Testing

Functional Testing	Validates all functional requirements of the system.
Performance Testing	Checks performance aspects like load, stress, scalability.
Security Testing	Ensures system is secure from threats and vulnerabilities.
Usability Testing	Checks ease of use and user experience.
Compatibility Testing	Verifies system works across different OS, browsers, devices.
Reliability Testing	Checks system stability and failure recovery.
Installation Testing	Verifies installation and uninstallation process.

Integration vs System Testing (At a Glance)

Aspect	Integration Testing	System Testing
What is tested?	Combination of modules	Complete integrated system
By whom?	Developers / Testers	Independent Test Team
Focus	Module interactions	End-to-end functionality
Test Approach	Incremental	Complete
Goal	Find defects in interfaces	Validate system against requirements
Example	DB module + API module	Entire application (e.g., e-commerce)

Integration & System Testing Flow (Quick View)



Best Practices

- Plan integration points early.
- Use stubs/drivers where needed.
- Automate integration & system tests.
- Use production-like environment for system testing.
- Maintain traceability with requirements.
- Log, track and fix defects promptly.
- Perform end-to-end scenarios.

12. Functional + Non-Functional Testing

@ codedsoul_05

What is Functional and Non-Functional Testing?

Functional Testing: Validates the system's functionality as per requirements and business rules.

Non-Functional Testing: Validates the system's quality attributes like performance, security, usability, reliability, etc.

> **Note :**
> Functional testing answers "Does it work?"
> Non-functional testing answers "How well does it work?" ☺

Functional Testing

- Focuses on WHAT the system does.
- Validates features, functions and business logic.
- Based on requirements and specifications.
- Usually performed at System / Acceptance level.
- Examples: Login, Add to Cart, Checkout, Search.

Non-Functional Testing

- Focuses on HOW WELL the system works.
- Validates quality attributes and constraints.
- Based on non-functional requirements.
- Can be done at any level.
- Examples: Load time, Security, Usability, Recovery.

Key Differences

Aspect	Functional Testing	Non-Functional Testing
Purpose	To verify the functionality and features.	To verify quality attributes and constraints.
Focus	What the system does.	How well the system works.
Based on	Functional Requirements / Business Rules	Non-Functional Requirements
Test Techniques	Black Box Techniques	Usually Black Box (some White Box)
Test Data	Business data	Large data volumes / Special conditions
Examples	Login, Save, Delete, Calculate	Performance, Security, Usability
User Perspective	End user / Business	End user, IT team, Operations
When Performed	Throughout SDLC (Unit to Acceptance)	From Integration / System till Production
Measurement	Pass / Fail	Measured with metrics & thresholds

Types of Functional Testing

Unit Testing	Tests individual units / components for expected output.
Integration Testing	Verifies interfaces and data flow between modules.
System Testing	Validates the complete integrated system against requirements.
Acceptance Testing (UAT)	Validates the system from end-user business perspective.
Smoke Testing	Basic check to ensure major functionalities are working.
Sanity Testing	Narrow testing to check specific functionalities after a change.

Types of Non-Functional Testing

Performance Testing	Checks response time, throughput, scalability and stability.
Security Testing	Verifies protection of data and system from threats and vulnerabilities.
Usability Testing	Checks ease of use, UI/UX and user-friendliness.
Reliability Testing	Checks system stability, error handling and recovery.
Compatibility Testing	Checks system compatibility across devices, browsers, OS, etc.
Installation Testing	Verifies installation and uninstallation process.
Scalability Testing	Checks system capability to handle increasing load.

Examples

Functional Testing Example (Login)	
Test Case	Expected Result
Valid Username & Password	Login Successful
Invalid Password	Error Message
Empty Username	Validation Message
Locked User	Account Locked Message

Non-Functional Testing Example (Performance)	
Scenario	Expected Result
100 Users	Response time < 2 sec
1000 Users	Response time < 4 sec
5000 Users	System handles load without failure
Peak Load	No crash, graceful degradation

Key Takeaway

- Functional = Correctness of features
- Non-Functional = Quality of the system
- Both are essential for delivering a complete and reliable product. ☺

Functional vs Non-Functional Testing (At a Glance)

Aspect	Functional Testing	Non-Functional Testing
What it checks	Functions, features, business rules	Quality attributes, performance, security, usability, etc.
Goal	Verify correctness	Verify quality and reliability
Outcome	Meets functional requirements	Meets quality expectations
User View	"Does it work?"	"How well does it work?"
Examples	Login, Add to Cart, Checkout	Load Time, Security, Usability, Recovery

Testing Approach



13. Defect / Bug Life Cycle

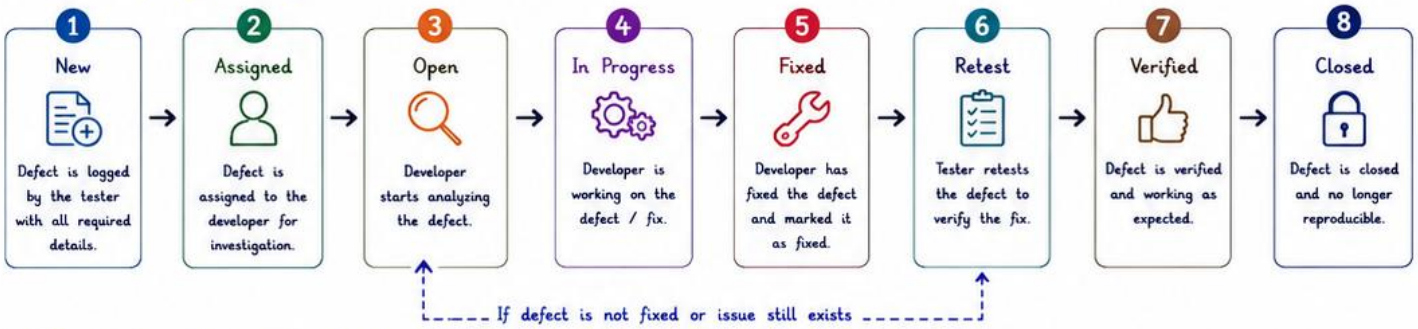
@ codedsoul_05

What is Defect / Bug Life Cycle?

Defect (or Bug) Life Cycle is the process that a defect follows from the time it is found by the tester until it is closed and verified.

Note :
A good defect life cycle ensures proper tracking, communication and quality improvement.

Defect Life Cycle Stages



Detailed Description of Each Stage

Stage	Description	Who Performs	Entry Criteria	Exit Criteria	Next Step
1 New	Defect is identified and logged in the defect tracking tool.	Tester	Issue found during testing.	Defect logged with all required details.	Assigned
2 Assigned	Defect is reviewed and assigned to the developer.	Test Lead / Manager	Defect is valid and reproducible.	Defect assigned to the right developer.	Open
3 Open	Developer acknowledges and starts analyzing the defect.	Developer	Defect assigned to developer.	Root cause identified / analysis started.	In Progress
4 In Progress	Developer works on the fix and makes code changes.	Developer	Cause of defect identified.	Code changes made and unit tested.	Fixed
5 Fixed	Developer has fixed the defect and marked it as fixed.	Developer	Code changes completed.	Defect marked as Fixed in the tool.	Retest
6 Retest	Tester retests the defect in the same environment.	Tester	Defect status is Fixed.	Fix verified or issue still exists.	Verified or Reopen
7 Verified	Tester confirms the fix is working as expected.	Tester / Test Lead	Defect retested successfully.	Defect works as expected and no side effects.	Closed
8 Closed	Defect is officially closed and archived.	Tester / Test Lead	Defect verified and accepted.	Defect closed in the tool.	End

Possible Additional Status (Loop Back)

Status	When it Happens
Reopen	If the fix does not work or the issue still exists.
Deferred	If the defect is not fixed in the current release.
Cannot Reproduce	If the tester is not able to reproduce the defect.
Duplicate	If the defect already exists in the system.
Rejected	If it is not a valid defect or working as designed.

Best Practices

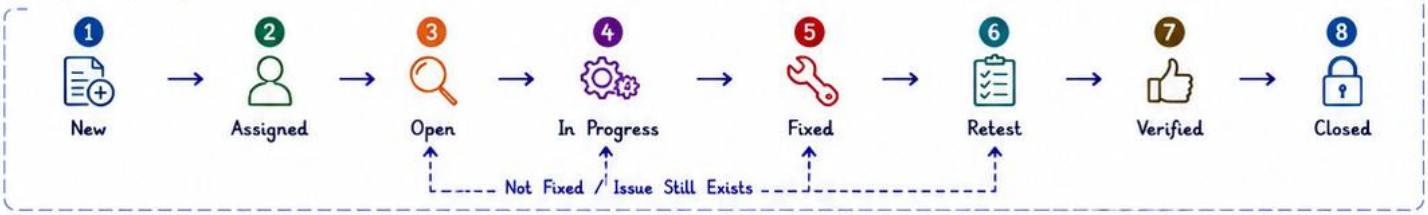
- Log clear and complete defect details.
- Use proper severity and priority.
- Reproduce before reporting.
- Communicate effectively.
- Retest in the same environment.
- Close only after proper verification.

Key Takeaway

A well-defined defect life cycle helps in:

- Better tracking
- Faster resolution
- Improved quality
- Good communication

Defect Life Cycle Flow (At a Glance)



14. Severity + Priority

@ codedsoul_05

What are Severity and Priority?

Severity: Indicates the impact of a defect on the application.
(How bad is the defect?)

Priority: Indicates the urgency to fix the defect.
(How soon should it be fixed?)

> **Note :**
> Severity is about the defect.
> Priority is about the fix.
> They are independent of each other.

Severity – Impact on the System

Level	Description	Impact	Example
Critical	System is unusable. Major functionality is broken.	System Crash, Data Loss	Application not starting
High	Major functionality not working as expected.	Major feature failure	Payment gateway failure
Medium	Minor functionality affected.	Partial impact, Workaround possible	UI misalignment, Report format issue
Low	Cosmetic issues, No impact on functionality.	No functional impact	Spelling mistake, Color issue

Priority – Urgency to Fix

Level	Description	When to Fix	Example
P1 – Urgent	Must be fixed immediately.	Before next build / release	Login not working
P2 – High	Should be fixed as soon as possible.	High priority in upcoming release	Invoice calculation incorrect
P3 – Medium	Can be fixed in normal release cycle.	Next or future release	UI enhancement request
P4 – Low	Fix when time permits.	Backlog / Future update	Typo in help text

Key Differences

Aspect	Severity	Priority
Purpose	Measures the impact of the defect on the system.	Indicates how soon the defect should be fixed.
Focus	On the defect itself.	On the resolution / business need.
Decided By	Tester / QA	Product Owner / Business / Stakeholders (in consultation with QA)
Scope	Technical impact.	Business impact and time sensitivity.
Can Change?	Usually remains same for a defect.	Can change based on release plan or business priority.
Example	System crash = Critical UI color issue = Low	System crash in production = P1 Urgent UI color issue before launch = P4 Low

Severity vs Priority – Example

Scenario	Severity	Priority	Reason
Login button not working	Critical	P1 – Urgent	System unusable, must fix immediately
Report page data miscalculated	High	P2 – High	Major functionality impact, fix soon
Alignment issue in dashboard	Low	P3 – Medium	Cosmetic issue, can wait
Typo in FAQ text	Low	P4 – Low	Minor issue, fix when time permits

Best Practices

- Always set both Severity and Priority for every defect.
- Do not use Severity to express your personal frustration.
- Be objective and consistent while assigning levels.
- Revisit and update Priority based on release plan.
- Communicate with team to align on definitions.

General Guidelines

- Not all Critical defects are P1.
  (E.g., Critical issue in rarely used feature may be P2)
- Not all High priority defects are Critical.
  (E.g., Spelling mistake before demo can be P1)
- Use organization-specific definitions if provided.
- Keep team aligned to avoid confusion.

Quick Tip

Think of it like this:

Severity = How much it hurts?

Priority = How fast we need to heal it?

Severity & Priority Matrix (Example)

Severity \ Priority	P1 – Urgent	P2 – High	P3 – Medium	P4 – Low
Critical	Fix Immediately	Fix Very Soon	Plan Soon	Plan Later
High	Fix Very Soon	Plan Soon	Plan in Next Release	Backlog
Medium	Plan Soon	Plan in Next Release	Backlog	Backlog
Low	Plan in Next Release	Backlog	Backlog	Backlog

Defect Handling Flow (High Level)



15. Test Automation Fundamentals

@ codedsoul_05

What is Test Automation?

Test Automation is the use of tools and scripts to execute test cases automatically, compare results, and generate reports.

- Saves time
- Improves accuracy
- Enables regression testing at scale

Note :
Automation is not about writing more scripts, it's about writing the right scripts for the right tests.

When to Automate?

- Repetitive test cases
- Stable & non-changing features
- Cross-browser / device testing
- Regression testing
- Large data-driven scenarios

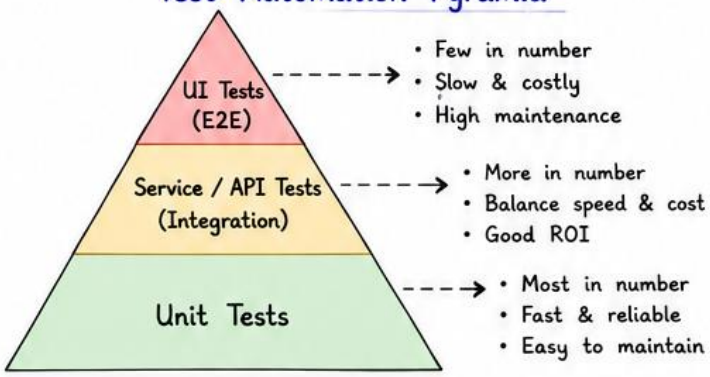
When NOT to Automate?

- New or frequently changing features
- One-time test scenarios
- Very small projects
- Exploratory / Ad-hoc testing
- UI changes too frequently

Popular Automation Tools

Selenium	Web Automation
Postman	API Testing
Cucumber	BDD Framework
Playwright	Modern Web Testing
JUnit	Unit Testing (Java)

Test Automation Pyramid



Key Components

Test Script	The code that runs the test
Test Data	Input data for test execution
Test Framework	Structure & guidelines (e.g., Page Object Model)
Test Environment	Where tests are executed (Dev, QA, Staging)
Reporting & Logs	Results, screenshots, logs

Manual vs Automation Testing

Aspect	Manual Testing	Automation Testing
Speed	Slower	Faster (after setup)
Accuracy	Human errors possible	More accurate
Cost	Low initial, High long-term	High initial, Low long-term
Best For	Exploratory, One-time tests	Regression, Repetitive tests
Maintenance	Easy	Requires effort
Execution	Manual effort	Script-driven

Types of Test Automation

- **UI Automation** – Testing UI of application (Web/Mobile/Desktop)
- **API Automation** – Testing APIs & Services
- **Database Testing** – Validate database operations
- **Mobile Automation** – Android / iOS testing
- **Performance Automation** – Load, Stress, Scale

Best Practices

- Follow Page Object Model (POM)
- Write reusable & maintainable scripts
- Use meaningful naming conventions
- Keep tests independent & atomic
- Handle waits, exceptions & retries
- Version control your test code (Git is your friend)

Popular Frameworks

Selenium (Web)	Playwright (Web)	Cypress (Web)	TestNG (Java)	pytest (Python)
• Java • Python • C#	• JavaScript • TypeScript • Python	• JavaScript • TypeScript	• Java	• Python

Automation Testing Flow (Quick View)

Analyze Requirement → Select Test Cases for Automation → Write / Maintain Test Scripts → Execute Tests → Review Reports & Fix Issues → Re-Run & Verify

Remember: Automate smarter, not harder. Focus on value, stability & ROI.

16 Selenium + Automation Concepts

@ codedsoul_05

1 What is Selenium?

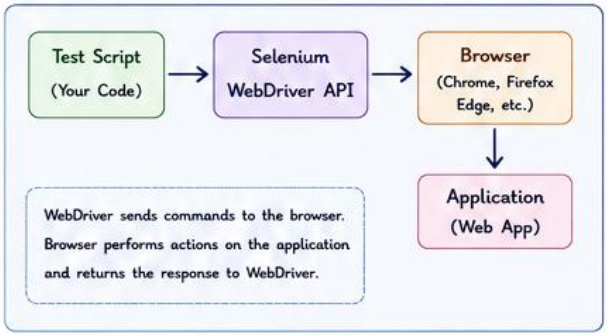
Selenium is an open-source suite of tools used for automating web browsers. It supports multiple programming languages and platforms.

Selenium Suite	
Selenium IDE	Record & playback tool for quick tests
Selenium WebDriver	Code-based automation for web apps
Selenium RC	(Deprecated) Older tool, not recommended
Selenium Grid	Run tests in parallel on multiple machines/browsers

Note

Selenium WebDriver is the most widely used tool for test automation nowadays.

2 How Selenium Works?



3 Selenium WebDriver Architecture



- WebDriver acts as a bridge between test script and browser.
- Browser Drivers are specific executables for each browser.
- Communication happens using JSON Wire Protocol / W3C Protocol.

4 Locators in Selenium

Locator	Syntax	Example
ID	`By.id("userId")`	`<input id="userId">`
Name	`By.name("username")`	`<input name="username">`
Class Name	`By.className("btn")`	`<button class="btn">`
Tag Name	`By.tagName("a")`	`<a href="#"></a>`
Link Text	`By.linkText("Click Here")`	`<a>Click Here</a>`
Partial Link Text	`By.partialLinkText("Click")`	`<a>Click Here</a>`
CSS Selector	`By.cssSelector("input[type='text']")`	`<input type="text">`
XPath	`By.xpath("//input[@id='userId']")`	`<input id="userId">`

5 Common WebDriver Commands

Action	Example (Java)
Launch Browser	`WebDriver driver = new ChromeDriver();`
Open URL	`driver.get("https://example.com");`
Find Element	`WebElement user = driver.findElement(By.id("userId"));`
Send Keys	`user.sendKeys("admin");`
Click	`driver.findElement(By.id("loginBtn")).click();`
Get Text	`String text = driver.findElement(By.id("msg")).getText();`
Get Attribute	`String value = user.getAttribute("value");`
Close Browser	`driver.quit();`

6 Waits in Selenium

Wait Type	Description	Example (Java)

Implicit Wait	Waits for a certain time for an element to be present.	`driver.manage()`
		`.timeouts()`
		`.implicitlyWait(10,`
		`TimeUnit.SECONDS);`

Explicit Wait	Waits until a condition is true.	`WebDriverWait wait =`
		`new WebDriverWait(driver,`
		`Duration.ofSeconds(10));`
		`wait.until(ExpectedConditions`
		`.visibilityOfElementLocated`
		`(By.id("userId"));`

Fluent Wait	Waits with polling frequency and ignores exceptions.	`Wait<WebDriver> wait =`
		`new FluentWait<>(driver)`
		`.withTimeout(Duration.ofSeconds(30))`
		`.pollingEvery(Duration.ofSeconds(5))`
		`.ignoring(NoSuchElementException.class);`

7 Handling Common Scenarios

Alerts	`Alert alert = driver.switchTo().alert();` `alert.accept(); // or alert.dismiss();`
Frames	`driver.switchTo().frame("frameName");` `driver.switchTo().defaultContent();`

Windows	`String parent = driver.getWindowHandle();`
	`for (String win : driver.getWindowHandles())`
	`if (!win.equals(parent)) driver.switchTo().window(win);`

Dropdown	`Select dropdown = new Select(driver.findElement(By.id("dd")));` `dropdown.selectByVisibleText("India");`
Checkbox	`WebElement chk = driver.findElement(By.id("chk"));` `if (!chk.isSelected()) chk.click();`

8 Test Automation Concepts

Reusability	Create reusable methods, libraries and data.
Maintainability	Write clean, modular and well-structured code.
Data Driven Testing	Use external data (Excel, CSV, DB) to run tests with multiple inputs.
Keyword Driven Testing	Use keywords/actions stored in a table or file.
Page Object Model (POM)	Represent each page as a class with locators and actions.
Logging	Log steps, info, warnings and errors.
Screenshots on Failure	Capture screenshot when test fails.
Reports	Generate meaningful and readable reports.

9 Selenium + Framework Integration

Framework	Use
TestNG / JUnit	For test organization, annotations, assertions and reporting.
Maven / Gradle	For build management and dependency handling.
Log4j / SLF4J	For logging the execution.
Apache POI	For reading/writing Excel files.
ExtentReports / Allure	For advanced test reporting.

10 Best Practices

- Follow Page Object Model for better maintainability.
- Use Explicit Wait instead of Thread.sleep().
- Keep locators stable and independent of UI changes.
- Use meaningful names for variables, methods and classes.
- Run tests in headless mode in CI/CD for faster execution.
- Keep test data separate from test scripts.



11 Popular Browser Drivers

- **ChromeDriver** — Automates Google Chrome
- **GeckoDriver** — Automates Mozilla Firefox
- **EdgeDriver** — Automates Microsoft Edge
- **SafariDriver** — Automates Safari (Mac)
- **OperaDriver** — Automates Opera Browser

17. API + Performance Testing

@codedsoul_05

1 What is API Testing?

API Testing is the process of testing Application Programming Interfaces to validate functionality, reliability, performance, security and error handling.

2 Why API Testing?

- Early testing in SDLC
- Faster than UI testing
- Finds issues in logic & data
- Service virtualization
- Cost-effective

3 Types of API Testing

Type	Description
Functional Testing	Validates API functionality as per requirements.
Reliability Testing	Checks API stability, availability and error handling.
Performance Testing	Validates response time, throughput and scalability.
Security Testing	Checks authorization, authentication and vulnerabilities.
Contract Testing	Ensures provider and consumer follow the same contract.

4 API Testing Process

1. Understand Requirements → 2. Design Test Cases → 3. Execute API Tests → 4. Validate Responses → 5. Report Defects

5 Key Elements in API Testing

Element	What to Validate
Endpoint (URL)	Correct URL and path
HTTP Method	GET, POST, PUT, DELETE, PATCH, etc.
Headers	Content-Type, Authorization, Accept, etc.
Request Body	Input data format, required fields, data types
Response Code	Status codes (200, 201, 400, 401, 404, 500, etc.)
Response Body	Correct data, structure, field values
Performance	Response time within acceptable limit
Security	Authentication, authorization, input validation

6 API Test Case Example (POST /login)

Test Scenario	Request (Sample)	Expected Result
Valid Login	{ "username": "user1", { "password": "Pass@123" }	Status: 200 OK Response contains token
Invalid Password	{ "username": "user1", { "password": "wrong@123" }	Status: 401 Unauthorized Error message
Missing Field	{ "username": "user1" }	Status: 400 Bad Request Validation error
Empty Body	{ }	Status: 400 Bad Request Required fields missing
SQL Injection	{ "username": " ' OR '1'='1", { "password": "anything" }	Status: 400 Bad Request Input validation works

7 Popular API Testing Tools

- Postman
- REST Assured
- SoapUI
- Newman (Postman CLI)
- JMeter (For Performance)

8 What is Performance Testing?

Performance Testing is a type of Non-Functional testing that evaluates an application's speed, responsiveness, stability and scalability under various workloads.

9 Goals of Performance Testing

- Identify Bottlenecks
- Ensure Scalability
- Improve Performance
- Ensure Reliability
- Better User Experience

10 Types of Performance Testing

Type	Description
Load Testing	Test system behavior under expected load.
Stress Testing	Test beyond normal capacity to find breaking point.
Spike Testing	Test sudden increase or decrease in load.
Endurance / Soak Testing	Test system stability under continuous load for long time.
Volume Testing	Test system with large amounts of data.
Scalability Testing	Test system's ability to scale up/down.
Baseline Testing	Test current performance to compare future changes.

11 Performance Testing Process

1. Define Objectives → 2. Plan Test Strategy → 3. Design Test Scenarios → 4. Execute Tests → 5. Analyze Results → 6. Report & Improve

12 Key Performance Metrics

Metric	Description
Response Time	Time taken by the system to respond.
Throughput	Number of requests processed per second/minute.
Concurrency	Number of users/requests at the same time.
Latency	Delay before the response starts.
TPS (Transactions per Second)	Number of transactions processed per second.
Error Rate	Percentage of failed requests.
Resource Utilization	CPU, Memory, Disk, Network usage.

13 Performance Test Scenario Example (Login API)

Test Type	Scenario	Users	Duration	Expected Result
Load Test	Normal expected load	100	30 min	Avg response time < 2 sec Error rate < 1%
Stress Test	Beyond capacity	1000	30 min	System handles till limit, no crash
Spike Test	Sudden load increase	100 → 1000	10 min	System stable, recovers quickly
Soak Test	Long duration load	200	8 hours	No memory leak, stable performance

14 Popular Performance Testing Tools

- Apache (Apache JMeter)
- LoadRunner (Micro Focus)
- k6 (Grafana)
- NeoLoad
- (Gatling.io)

15 API Testing vs Performance Testing

Aspect	API Testing	Performance Testing
Purpose	Validate functionality, reliability, security of APIs	Validate speed, stability, scalability under load
Focus	Correct response, status codes, data validation	Response time, throughput, resource usage
Level	Functional + Non-Functional	Non-Functional
Tools	Postman, Rest Assured, SoapUI	JMeter, LoadRunner, k6, NeoLoad
When Performed	Early in SDLC (after development)	Later in SDLC (after system is stable)
Users Involved	Developers, Testers	Performance Testers, DevOps
Outcome	Quality of API	Performance & Capacity of System

16 Best Practices

- Understand API documentation thoroughly.
- Validate all status codes and error scenarios.
- Use environment and data driven approach.
- Automate API tests and integrate with CI/CD.
- For performance tests, start with baseline and analyze.
- Monitor system resources during performance tests.
- Identify bottlenecks and work with dev team.

17 Common Challenges

- Lack of proper API documentation
- Dynamic data and authorization handling
- Environment configuration issues
- Flaky tests due to external dependencies
- Identifying real bottlenecks
- High resource cost for performance tests

18 Key Takeaway

- API Testing ensures APIs work correctly.
- Performance Testing ensures system works well under load.
- Both are essential for delivering high quality, fast and reliable applications.
- Automate early, measure continuously, improve always!

18. Agile Testing + CI/CD

@codedsoul_05

1 What is Agile Testing?

Agile Testing is the practice of testing in Agile teams to deliver quality software iteratively and incrementally. It focuses on collaboration, continuous feedback and early defect detection.



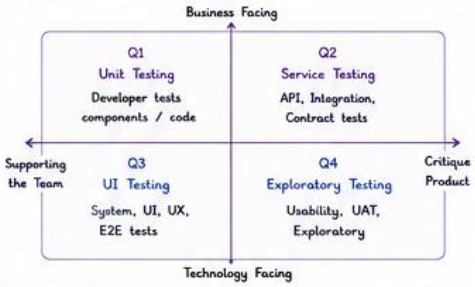
2 Principles of Agile Testing

- Whole team approach
- Early and continuous testing
- Shift-left testing
- Customer collaboration
- Respond to change
- Quality in every iteration

Note :

Agile Testing is not a phase, it's a mindset!

3 Agile Testing Quadrants



4 Agile Testing Activities in STLC

STLC Phase	Agile Testing Activities
Requirement	Review user stories, write acceptance criteria, create test ideas
Planning	Estimate testing effort, define DoR, DoD, test strategy
Design	Create test cases, test data, automation strategy
Development	Unit testing, Code reviews, Static analysis
Testing	Functional, Non-functional, Regression, Exploratory
Release	UAT, Production validation, Release readiness
Feedback	Collect feedback, Retrospective, Improve

5 Agile Testing Best Practices

- Involve testers from the start.
- Keep test cases concise and to the point.
- Automate smartly and early.
- Use exploratory testing in every iteration.
- Focus on quality, not just on defects.
- Continuous feedback and improvement.

6 Benefits of Agile Testing

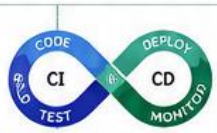
- Early defect detection
- Faster feedback
- Better collaboration
- High quality software
- Adaptable to changes
- Customer satisfaction

7 What is CI/CD?

CI (Continuous Integration) and CD (Continuous Delivery/Deployment) are practices that automate the integration, testing and delivery of code changes to deliver high-quality software faster and reliably.

Continuous Integration (CI)

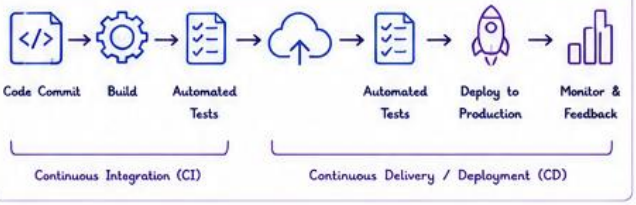
Developers integrate code frequently into a shared repository. Automated builds and tests validate the changes.



Continuous Delivery/Deployment (CD)

Code changes are automatically built, tested and deployed to staging or production environments with minimal manual intervention.

8 CI/CD Pipeline Flow



9 Key Differences

Aspect	Agile Testing	CI	CD (Delivery)	CD (Deployment)
Focus	Quality in Agile process	Integrate code frequently	Deliverable ready for release	Auto deploy to production
Goal	Deliver quality in every iteration	Detect integration issues early	Ensure release readiness	Release to users automatically
Trigger	Sprint / Iteration	Code commit	After successful testing	After approval / fully automated
Human Involvement	High Collaboration	Low (Automated)	Low (Automated)	Very Low (Automated)
Output	Tested increments	Integrated build	Release candidate	Live application
Tools (Examples)	Jira, Zephyr, Xray	Jenkins, GitLab CI, CircleCI	Jenkins, GitLab CI, Azure DevOps	Jenkins, Argo CD, Spinnaker

10 Role of QA in Agile + CI/CD

- Work with BA/PO to clarify requirements
- Review stories and define acceptance criteria
- Create automated tests and maintain test suites
- Participate in CI pipeline and ensure quality gates
- Perform exploratory testing in each iteration
- Monitor releases and provide feedback

Quality Gates

Quality Gates are checks in the pipeline that must be passed for the build to proceed further.

11 Test Automation in CI/CD

- Automated tests run at every stage (Unit, API, UI, Regression).
- Fast feedback to developers.
- Reduces manual effort and human error.
- Enables continuous testing and faster releases.



12 Typical CI/CD Tools

Code Repository	CI Tools	Build Tools	Test Tools	Deployment Tools	Monitoring Tools
GitHub	Jenkins	Maven	Selenium	Docker	Prometheus
GitLab	GitLab CI	Gradle	JUnit	Kubernetes	Grafana
Bitbucket	CircleCI	Ant	TestNG	Helm	ELK Stack
			Postman		
			JMeter		

13 Agile Testing vs Traditional Testing

Aspect	Agile Testing	Traditional Testing
Approach	Iterative & Incremental	Phase-based (Waterfall)
Testing Start	Early & Continuous	After Development
Team	Whole Team Involved	Separate Testing Team
Test Planning	Flexible, Adapts to Change	Rigid, Fixed Plan
Documentation	Minimal, Just Enough	Comprehensive
Delivery	Frequent Releases	Single Release
Customer Involvement	High	Low

14 Example: CI/CD Pipeline for Web Application

	Step	Description	Tools (Example)
1	Code Commit	Developer commits code to repository	GitHub / GitLab
2	Build	Compile code and create build artifact	Maven / Gradle
3	Unit Test	Run unit tests	JUnit / TestNG
4	Code Quality Check	Static code analysis	SonarQube
5	Integration Test	Run API / Service tests	Postman / RestAssured
6	UI Test	Run UI automation tests	Selenium / Cypress
7	Deploy to Staging	Deploy build to staging environment	Docker / Kubernetes
8	Acceptance Test	Run acceptance / E2E tests	Selenium / Postman
9	Deploy to Production	Deploy to production environment	Jenkins / Argo CD
10	Monitor	Monitor application & collect feedback	Prometheus / Grafana

15 Challenges

- Frequent changes in requirements
- Test automation maintenance
- Test data management
- Environment setup & stability
- Skill set and mindset shift
- Time pressure in every sprint



16 Best Practices

- Follow Shift-left approach.
- Automate the right tests.
- Keep test data and environment stable.
- Maintain independent, reusable tests.
- Use CI/CD quality gates effectively.
- Continuously monitor and improve.



17 Key Takeaway

Agile Testing ensures quality throughout the Agile journey, while CI/CD automates the flow from code to customer.

Together they enable faster delivery, higher quality and happier customers!

Test Early • Test Often • Test Smart • Deliver Faster

19. REAL-WORLD TESTING PROBLEMS

@codedsoul_05

Common Scenarios, Challenges & How Testers Think

How to Use This — Read the scenario → Understand the problem → Think of test strategy → Identify risks → Suggest testing approach

1 E-commerce Checkout Fails

Scenario: Users are unable to place orders when using a specific payment method.

What to Test

- Payment flow end-to-end
- Different payment modes
- Valid/invalid card details
- Gateway timeout / decline cases
- Order creation & confirmation

Risks

Revenue loss, poor user experience, loss of customer trust

→ Approach

Functional + Integration + Negative + Regression testing on payment flow.

2 App Crashes on Upload

Scenario: Mobile app crashes while uploading large files or images.

What to Test

- Different file sizes & formats
- Poor / unstable network
- Background / foreground transitions
- Memory usage & app stability

Risks

Data loss, poor user experience, negative app ratings

→ Approach

Functional + Reliability + Performance + Compatibility testing.

3 Slow Performance During Peak Hours

Scenario: Application is slow during high traffic or peak usage times.

What to Test

- Load testing
- Response time under heavy load
- Database performance
- Server resource utilization

Risks

Slow app, timeouts, user drop-off, business impact

→ Approach

Performance testing (Load, Stress, Spike) + Monitoring + Bottleneck analysis.

4 Users See Someone Else's Data

Scenario: Some users are able to view other users' data occasionally.

What to Test

- Authentication & authorization
- Session management
- Data isolation between users
- API security & access control

Risks

Data breach, privacy issues, legal & compliance impact

→ Approach

Security testing + Functional + API testing + Session management validation.

5 Feature Works in Dev, Fails in Prod

Scenario: A feature works perfectly in development but fails after deployment.

What to Test

- Environment differences
- Configuration issues
- Database/schema changes
- Third-party service integration

Risks

Production failure, downtime, loss of user trust

→ Approach

Smoke + Sanity + Environment validation + Regression testing in production-like env.

6 Notifications Not Received

Scenario: Users are not receiving email or push notifications.

What to Test

- Notification triggers
- Email / SMS / Push configurations
- Third-party service status
- Spam / Promotions folders (email)

Risks

Missed important info, poor engagement, user dissatisfaction

→ Approach

Functional + Integration + End-to-end + Negative testing.

7 Data Not Saved Sometimes

Scenario: Data entered by users is not saved sometimes.

What to Test

- Database transactions
- Network interruptions
- Form validations
- Concurrency (multiple users)

Risks

Data loss, inconsistent data, user frustration

→ Approach

Functional + Database + Concurrency + Recovery testing.

8 UI Looks Broken on Some Devices

Scenario: UI is not displaying properly on certain devices or screen sizes.

What to Test

- Responsive design
- Cross-browser compatibility
- Different screen resolutions
- UI elements alignment

Risks

Poor user experience, usability issues, brand impact

→ Approach

Compatibility + Responsive + UI testing on multiple devices & browsers.

9 Password Reset Not Working

Scenario: Users are not able to reset their password via email link.

What to Test

- Reset link generation
- Email delivery
- Link expiry
- Reset flow & validations

Risks

Users get locked out, support tickets, security concerns

→ Approach

End-to-end + Functional + Security + Negative testing.

10 General Real-World Challenges

Challenge	How Testers Handle It
Unclear Requirements	Ask questions, create test charter, collaborate with BA/Dev
Tight Deadlines	Prioritize tests, risk-based testing, automate critical flows
Frequent Changes	Use Agile approach, regression automation, maintain test cases
Test Data Issues	Create good data, use data factories, mask sensitive data
Environment Instability	Use stable test environments, service virtualization, mock dependencies
Flaky Tests	Identify root cause, stabilize tests, avoid brittle locators

11 How to Think Like a Tester?

- Understand the workflow end-to-end
- Think from the user's perspective
- Identify what can go wrong
- Consider data, validations & rules
- Check boundaries & error scenarios
- Explore, don't just follow test cases
- Use risk-based approach
- Always verify fixes & prevent re-opening

12 Key Takeaways

- Real-world problems have no single correct answer.
- Understand the business impact.
- Identify risks and prioritize testing.
- Combine different types of testing.
- Communicate, collaborate and keep learning.

A good tester finds problems before users do!

Tester's Mantra: Question Everything | Think Deep | Test Smart | Deliver Quality

20. Top 100 Software Testing Interview Questions + Quick Revision Cheat Sheet

Prepare Smart' Crack Any Interview!

TOP 100 SOFTWARE TESTING INTERVIEW QUESTIONS

1 Testing Fundamentals (1–10)

1. What is Software Testing?
2. Why is testing important?
3. What are the different testing levels?
4. What is the V-Model?
5. What is STLC?
6. What is the difference between verification and validation?
7. What is the difference between error, defect and failure?
8. What is a test case?
9. What are the components of a good test case?
10. What is Test Scenario?

2 SDLC & STLC (11–20)

11. What are different SDLC models?
12. What is Waterfall model?
13. What is Agile model?
14. What is the difference between SDLC and STLC?
15. What are the phases of STLC?
16. What happens in Requirement Analysis phase?
17. What is Test Plan?
18. What is Test Strategy?
19. What is Traceability Matrix?
20. What is Test Closure Report?

3 Test Design Techniques (21–30)

21. What is Test Design?
22. What are different test design techniques?
23. What is equivalence partitioning?
24. What is boundary value analysis?
25. What is decision table testing?
26. What is state transition testing?
27. What is use case testing?
28. What is error guessing?
29. What is exploratory testing?
30. What is pairwise testing?

4 Test Levels (31–40)

31. What is Unit Testing?
32. What is Integration Testing?
33. What is System Testing?
34. What is Acceptance Testing?
35. What is Smoke Testing?
36. What is Sanity Testing?
37. What is Regression Testing?
38. What is Re-Testing?
39. What is End-to-End Testing?
40. What is Alpha and Beta Testing?

5 Test Types (41–50)

41. What is Functional Testing?
42. What is Non-Functional Testing?
43. What is UI Testing?
44. What is Usability Testing?
45. What is Compatibility Testing?
46. What is Installation Testing?
47. What is Performance Testing?
48. What is Security Testing?
49. What is Localization Testing?
50. What is Accessibility Testing?

6 Test Management (51–60)

51. What is Test Estimation?
52. What is Test Planning?
53. What are Test Metrics?
54. What are entry and exit criteria?
55. What is Defect/Bug?
56. What is Defect Life Cycle?
57. What is Severity and Priority?
58. What is Bug Report?
59. What is Test Monitoring?
60. What is Test Control?

7 Defects (61–70)

61. What is Defect Life Cycle?
62. What are different defect statuses?
63. What is severity?
64. What is priority?
65. What is the difference between severity and priority?
66. Who can change the priority of a defect?
67. What is RCA (Root Cause Analysis)?
68. What is a blocker defect?
69. What is a critical defect?
70. What is defect leakage?

8 Tools & Automation (71–80)

71. What is Test Automation?
72. What are the benefits of automation?
73. What are the types of test automation?
74. What is Selenium?
75. What are WebDriver components?
76. What is a Locator in Selenium?
77. What is TestNG?
78. What is POM (Page Object Model)?
79. What is CI/CD?
80. What are the tools used in CI/CD?

9 API Testing (81–90)

81. What is an API?
82. What is API testing?
83. What are types of API testing?
84. What is REST?
85. What is SOAP?
86. What are HTTP methods?
87. What is status code in API testing?
88. What is Postman?
89. How do you validate an API response?
90. What is contract testing?

10 Real-World & Scenario Based (91–100)

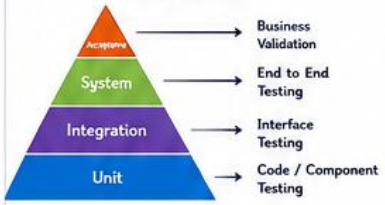
91. How would you test a login feature?
92. How would you test an e-commence application?
93. How do you test with limited time?
94. How do you prioritize test cases?
95. How do you handle tight deadlines?
96. What will you do if requirements keep changing?
97. How do you ensure good test coverage?
98. How do you test in Agile?
99. What are challenges in testing?
100. What is the most important quality of a tester?

QUICK REVISION CHEAT SHEET

1. STLC PHASES

1. Requirement Analysis
2. Test Planning
3. Test Case Development
4. Environment Setup
5. Test Execution
6. Test Cycle Closure

2. TEST LEVELS



3. TEST TYPES

Functional	Non-Functional
• Functional	• Performance
• UI	• Security
• Usability	• Reliability
• Compatibility	• Scalability
• Installation	• Usability
• Localization	• Maintainability
• Accessibility	• Portability

4. DEFECT LIFE CYCLE

New → Assigned → Open → Fixed → Retest → Closed

Open → Deferred

Retest → Reopen

5. SEVERITY vs PRIORITY

SEVERITY (Impact)

- Critical – System crash
- Major – Major feature not working
- Minor – Minor issue
- Trivial – Cosmetic issue

PRIORITY (Urgency)

- P1 – Fix Immediately
- P2 – Fix Soon
- P3 – Fix Later
- P4 – Fix if time permits

6. TEST DESIGN TECHNIQUES

Equivalence Partitioning | Boundary Value Analysis | Decision Table Testing | State Transition Testing | Use Case Testing | Error Guessing

7. COMMON HTTP STATUS CODES

200 – OK (Success)
201 – Created
400 – Bad Request
401 – Unauthorized
403 – Forbidden
404 – Not Found
500 – Internal Server Error
503 – Service Unavailable

8. HTTP METHODS

GET – Retrieve data
POST – Create data
PUT – Update data
DELETE – Delete data
PATCH – Partial update

API

9. TEST AUTOMATION PYRAMID



- ✓ More UI tests are costly & slow
- ✓ More API tests bring better ROI
- ✓ More Unit tests bring stability

10. AGILE TESTING PRINCIPLES

- ✓ Whole team approach
- ✓ Shift-left testing
- ✓ Continuous testing
- ✓ Test early, test often
- ✓ Automate smartly
- ✓ Respond to change
- ✓ Quality in every iteration

11. CI/CD PIPELINE (TYPICAL FLOW)

Code Commit → Build → Automated Tests → Deploy → Monitor → Feedback

12. BUG REPORT TEMPLATE

Title:
Module:
Description:
Steps to Reproduce:
Expected Result:
Actual Result:
Severity:
Priority:
Environment:
Attachments:

13. TEST CASE TEMPLATE

Test Case ID:
Test Scenario:
Test Case Description:
Pre-Conditions:
Test Steps:
Test Data:
Expected Result:
Post-Conditions:
Status:

14. GOLDEN RULES OF TESTING

- ✓ Test shows presence of defects, not absence.
- ✓ Exhaustive testing is impossible.
- ✓ Start testing early.
- ✓ Defects cluster together.
- ✓ Pesticide paradox – keep reviewing tests.
- ✓ Testing depends on context.
- ✓ Absence-of-errors fallacy.
- ✓ Test the right thing.
- ✓ Good testing saves time & money.
- ✓ Quality is everyone's responsibility.

INTERVIEW ★ TIPS

Understand the concepts clearly, don't mug up.
Give real examples from your experience.
Communicate clearly and confidently.
Stay calm, take a moment to think.
Practice more mock interviews.

★ Learn – Practice – Revise – Repeat – Success is Inevitable!

All the Best for Your Interview!